CS19541 - COMPUTER NETWORKS LAB MANUAL

EXPERIMENT 14

AIM: To implement a Packet Sniffer using raw sockets in Python to capture and inspect Ethernet, IP, and TCP/UDP header details of network traffic.

DESCRIPTION:

- **Raw Socket Packet Sniffing:** A packet sniffer intercepts network traffic passing through a network interface by binding a raw socket directly to network layer frames, allowing deep inspection of headers across different OSI protocol layers.
- **Header Parsing:** Raw byte streams captured from the network socket are unpacked into structured packet headers including Ethernet frames (MAC addresses), IPv4 headers (source/destination IP, TTL, protocol), and transport headers (TCP/UDP ports).

FEATURES AND CAPABILITIES:

What we can do with Raw Socket Packet Sniffers:

- Capture raw network frames directly from physical or loopback network interfaces.

- Parse Ethernet frame headers to extract source and destination MAC addresses and EtherType protocol codes.
- Extract IPv4 header fields including version, header length, Time to Live (TTL), protocol identification, and IP addresses.
- Unpack TCP/UDP transport protocol headers to inspect source/destination ports, sequence numbers, and packet payloads.

PROGRAM ARCHITECTURE OVERVIEW:

Packet Sniffer Processing Architecture:

- **Socket Setup:** Initialize Raw Socket (AF_PACKET / AF_INET, SOCK_RAW) with administrative/root privileges.
- **Capture & Unpack Loop:** Receive Raw Frame (recvfrom) -> Unpack Ethernet Frame -> Unpack IP Header -> Unpack Transport (TCP/UDP) Header -> Display Protocol Analysis.

PROCEDURE & IMPLEMENTATION CODE

Build Packet Sniffer Application using Raw Sockets

1. Initialize a raw socket with socket.AF_PACKET (Linux) or socket.AF_INET to capture raw packet data.
2. Define parsing functions using the struct module to decode binary headers into human-readable values.
3. Continuously listen for incoming frames, parse multi-layer headers, and print detailed protocol info.

Packet Sniffer Code (packet_sniffer.py):

```python
import socket
```

```
import struct
import textwrap

def parse_ethernet_frame(data):
    # Placeholder for unpacking Ethernet header (MAC addresses &
Protocol)
    pass

def parse_ip_header(data):
    # Placeholder for unpacking IPv4 header (Version, TTL,
Protocol, IP Addresses)
    pass

def parse_tcp_header(data):
    # Placeholder for unpacking TCP header (Ports, Sequence, Flags)
    pass

def parse_udp_header(data):
    # Placeholder for unpacking UDP header (Ports, Length)
    pass

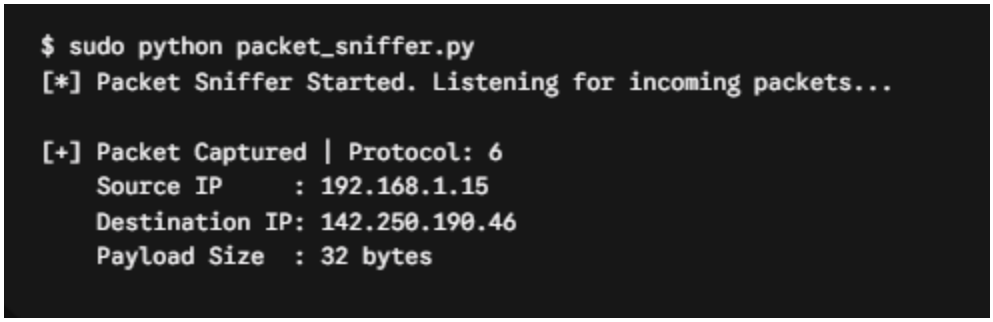
def main():
    # Main loop for sniffing network packets
    print("Starting Raw Socket Packet Sniffer...")

if __name__ == '__main__':
    main()
```

TERMINAL EXECUTION OUTPUT

Terminal Execution Output :

Execute the packet sniffer script with elevated administrative privileges to capture live network traffic and inspect Ethernet, IP, TCP, and UDP packet details.



Result:

Thus the experiment to implement a Packet Sniffer using raw sockets in Python to capture and inspect network traffic headers has been studied and executed successfully.